Design and Analysis of Algorithm (DAA), Lab-08

BTech (CS-B and CE), 3rd Semester
September 29, 2026



Instructor: Dr. Ajaya Kumar Dash.

Questions based on Dynamic Programming design paradigm
(Difficulty level: medium and hard)

1. **[Minimum Coin Change]** Given an integer array of coin denominations $C = \{c_1, c_2, \ldots, c_n\}$ representing coins of different values, and an integer target amount V, find the minimum number of coins needed to make up that amount. You may assume an infinite supply of each coin denomination. If that amount of money cannot be made up by any combination of the coins, return -1.

   By choosing the proper input representation, write a program in C to validate your algorithm and derive the complexity analysis of your algorithm.

2. **[Coin Change:Total number of ways]** Given an array of distinct positive integers representing coin denominations $C = \{c_1, c_2, \ldots, c_n\}$ and a target amount V, find the total number of distinct combinations of coins that sum up to V. You may assume an infinite supply of each coin denomination. The order of coins does not matter (e.g., $1 + 2$ and $2 + 1$ are considered the same combination). By choosing the proper input representation, write a program in C to validate your algorithm and derive the complexity analysis of your algorithm.

3. **[Longest Common Subsequence (LCS)]** Given two sequences $X = \langle x_1, x_2, \ldots, x_m \rangle$ and $Y = \langle y_1, y_2, \ldots, y_n \rangle$, compute the length of their longest common subsequence and reconstruct the actual subsequence string. By choosing the proper input representation, write a program in C to validate your algorithm and derive the complexity analysis of your algorithm.

4. **[Longest Increasing Subsequence]** Given an integer array $A = [a_0, a_1, \ldots, a_{n-1}]$, find the length of the longest subsequence such that all elements of the subsequence are strictly increasing. By choosing the proper input representation, write a program in C to validate your algorithm and derive the complexity analysis of your algorithm.

5. **[Maximum Sum Increasing Subsequence]** Given an array of n positive integers $A = [a_0, a_1, \ldots, a_{n-1}]$, find the maximum possible sum of a strictly increasing subsequence. By choosing the proper input representation, write a program in C to validate your algorithm and derive the complexity analysis of your algorithm.

6. **[Edit Distance with Traceback Information]** Given two strings A of length m and B of length n, compute the minimum number of operations (insertions, deletions, or substitutions) required to transform A into B, and print the traceback result. By choosing the proper input representation, write a program in C to validate your algorithm and derive the complexity analysis of your algorithm.

7. **[Rod Cutting with Reconstruction]** Given a rod of length n inches and an array of prices $P = [p_1, p_2, \ldots, p_n]$, where p_i denotes the market price of a rod piece of length i inches, determine:

   (i) The maximum revenue obtainable by cutting up the rod and selling the pieces.

   (ii) The exact lengths of the pieces that constitute the optimal decomposition (reconstruction).

   Cuts are integral and can be made in any combination (including leaving the rod uncut), and the sum of the piece lengths must equal n. By choosing the proper input representation, write a program in C to validate your algorithm and derive the complexity analysis of your algorithm.

8. **[Optimal Binary Search Trees (OBST)]** Given a set of n distinct sorted keys $K = \langle k_1, k_2, \ldots, k_n \rangle$ with search probabilities $p_1, p_2, \ldots, p_n$, and $n+1$ dummy keys $d_0, d_1, \ldots, d_n$ representing searches not in K with probabilities $q_0, q_1, \ldots, q_n$, find the minimum expected search cost of a binary search tree. By choosing the proper input representation, write a program in C to validate your procedures and derive the complexity analysis of your algorithm.

9. **[Collatz Conjecture]** (Open Unsolved Problem): The Collatz Conjecture (also known as the $3n+1$ problem or Ulam conjecture) defines a recurrence relation for any strictly positive integer n:

$$T(n) = \begin{cases} \frac{n}{2} & \text{if } n \text{ is even} \\ 3n+1 & \text{if } n \text{ is odd} \end{cases}$$

   The sequence repeatedly applies this function until $n = 1$. Although unproven and still an open problem, it is conjectured that the sequence reaches 1 for all positive integers. Write a modular C program to analyse the trajectory of a user-provided starting value $n \geq 1$ and across an interval $[a, b]$.

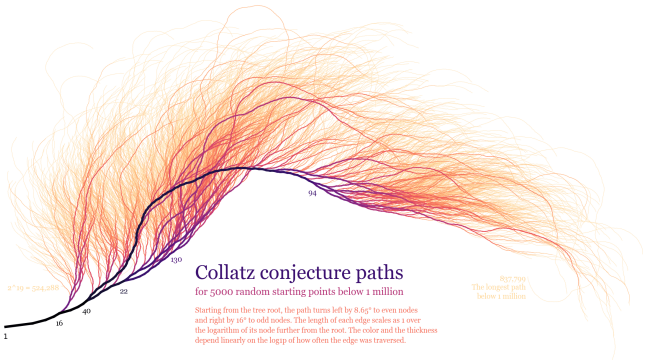


Figure 1: Collatz conjecture paths for 5000 random starting points below 10^6 (Ref: wiki).

Objective to give problem 9 as an assignment: To implement iterative control structures, functional decomposition, dynamic memory allocation/pointers, and integer overflow handling in C by simulating and analysing arithmetic trajectories defined by the Collatz Conjecture. Most importantly, to experience a so simple yet profound problem which is still open!! The simplicity of this problem is deceptive. The reason it remains open may lie in foundational issues in number theory, dynamical systems, and computability.
